COURSE PROJECT / OCTOBER 21, 2026 / ENGLISH EDITION

Virtual piano with closed shapes and MediaPipe Hands

Core implementation and classroom demonstration. A desktop browser turns camera-tracked fingertips into notes. MediaPipe supplies landmarks; project code maps them to closed key regions, confirms hover, manages shared note ownership, and plays sampled piano with visual feedback.

The app has 13 keys from C4 to C5, two-hand tracking, free play, and four teaching songs. It supports preview, section loops, tricky-section retries, and local history. Interaction is a two-dimensional image hit, not detection of real screen or table contact.

Questions the demonstration answers

- How do camera fingers become coordinates?

- How do coordinates align with keys and avoid edge mistakes?

- Why does holding attack once, and how do multiple fingers form chords?

- How do scores drive targets, assessment, and independent previews?

Area	Recorded status
Stack	TypeScript, Canvas 2D, Web Worker, MediaPipe, Web Audio
Historical automation	36 unit and 19 browser checks passed in the recorded environment
Continuous replay	15 minutes for the original 12-key version; not repeated for the current version
Manual acceptance	Physical occlusion, human chords, audible quality, end-to-end latency pending

Code excerpts are teaching-oriented and omit some UI/error branches. Function names, parameters, and rules correspond to the implementation. Source comments and project documentation are now in English. Historical results are preserved, not presented as new measurements.

OVERVIEW / OPERATION AND SCOPE

Features and operation

Watch the mirrored view and move fingertips over keys. Index fingers are the default. A 100 ms confirmed hover attacks a note; exit releases it, and re-entry can repeat it.

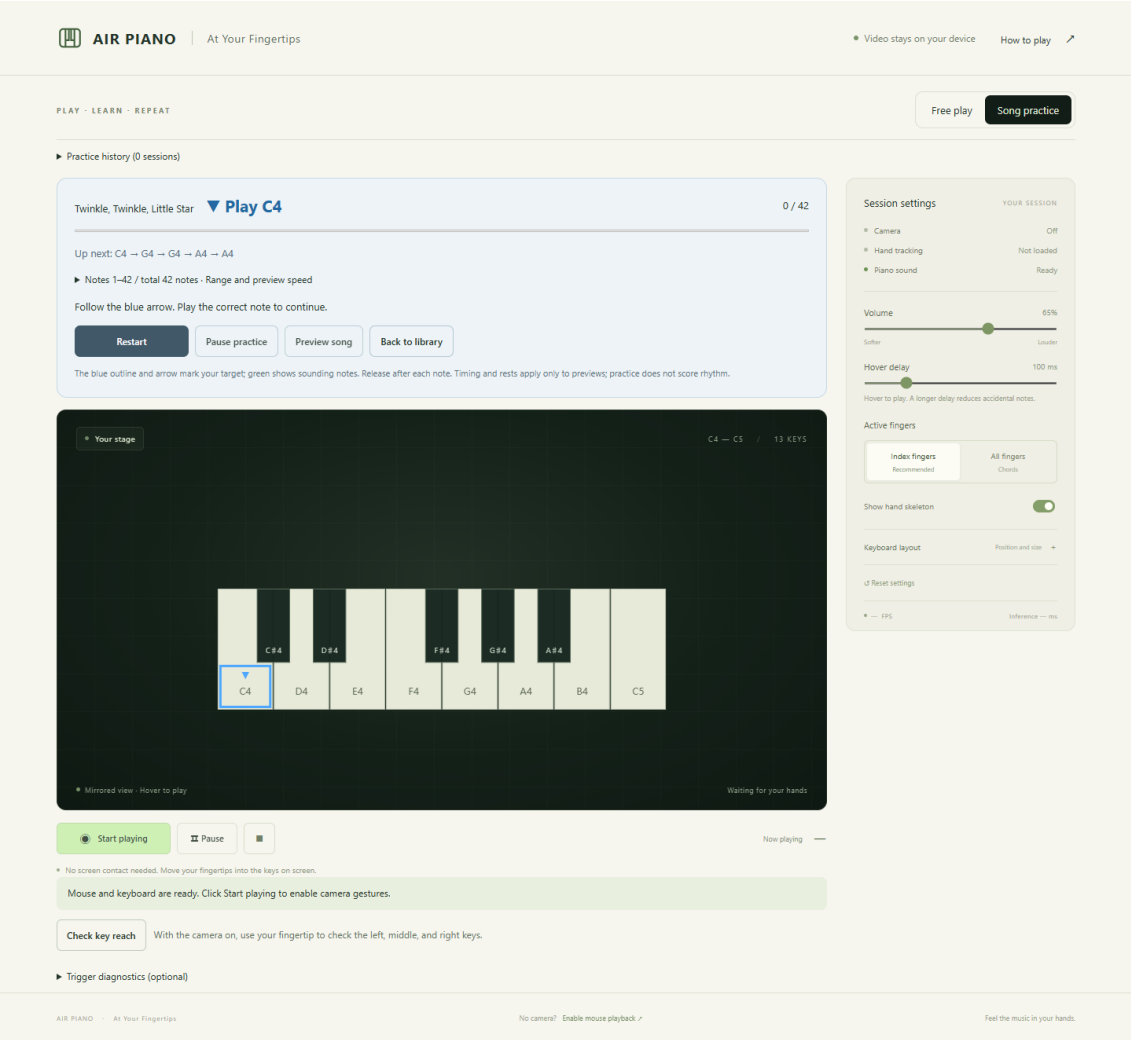


Figure 1. English note-practice view. The library collapses and the blue outline marks the target.

Action	Behavior
Start playing	Prepare camera, model, and audio together; allow input only when all are ready
Pause / Resume	Release sound and retain camera; reconfirm fingertips on resume
End session	Terminate worker, stop tracks, close audio
Mouse / keyboard	Practice and test audio without a camera
Song practice	Four teaching songs; correct advances, mistakes wait, repeats require release

SYSTEM DESIGN / DATA FLOW

Architecture and module responsibilities

A worker is a browser background thread. It performs synchronous recognition while the main thread handles UI, rules, and audio scheduling. Frames stay local and resources are hosted with the site; no business backend is required.



Figure 2. Video-to-landmark flow. The worker supplies positions; project code determines musical interaction.



Main-thread rules feed audio and visual feedback.

File	Responsibility	Entry points
main.ts	UI, orchestration, drawing	start / onResult / draw
tracking.ts	Camera/worker bridge	openCamera / capture / stop
tracker.worker.js	Model and video inference	init / frame
core.ts	Hits, timing, identity, ownership	HoverEngine / NoteOwners
audio.ts	Samples, pitch, envelope, cleanup	prepare / on / off
songs.ts / practice.ts	Scores, sections, history validation	validateSong / songSection
lesson.ts	Assessment, errors, timing	start / input / pause
song-player.ts	Audio-clock preview/cancel	play / stop

Valid user on/off events feed both sound and Lesson. Lesson updates targets and progress; Canvas draws the outline. Preview uses a separate channel and cannot advance practice. Rule modules are independently testable.

CORE TECHNIQUE 1 / FRESH FRAMES

Camera access and frame capture

Start prepares three resources concurrently

```
await Promise.all([
  tracker.openCamera(),
  tracker.initModel(),
  audio.prepare(),
]);
mode = "running";
```

Promise.all waits for camera, model, and audio. Camera success alone does not establish readiness. Any failure enters cleanup.

Connect a camera stream to the page

```
const stream = await navigator.mediaDevices.getUserMedia({
  video: { width: { ideal: 640 }, height: { ideal: 480 },
           facingMode: "user" },
  audio: false,
});
this.video.srcObject = stream;
await this.video.play();
```

getUserMedia returns a continuing stream. ideal requests rather than guarantees resolution; audio:false avoids microphone permission. The video element receives the stream through srcObject. Camera access requires localhost or HTTPS.

Submit only frames that can remain fresh

capture skips when a task is busy, video is not ready, the video time is unchanged, or fewer than 30 ms have passed. createImageBitmap scales to width 640 while preserving aspect ratio.

pending spans capture and inference, keeping at most one task in flight. postMessage transfers the image to the worker. Completion releases the busy flag. The system skips old opportunities instead of building a queue.

CORE TECHNIQUE 2 / SEPARATE INFERENCE

MediaPipe inference and worker communication



Figure 3. frame includes bitmap and timestamp; result returns landmarks and timing.

```
const options = delegate => ({
  baseOptions: { modelAssetPath, delegate },
  canvas: new OffscreenCanvas(1, 1),
  runningMode: "VIDEO", numHands: 2,
  minHandDetectionConfidence: 0.55,
  minHandPresenceConfidence: 0.55,
  minTrackingConfidence: 0.55,
});
```

The pretrained Hand Landmarker returns at most two hands. Initialization tries GPU, then CPU on failure. Detection, presence, and tracking thresholds are separate filters, not a claim of 55% accuracy. [1]

Inference and cleanup

detectForVideo(bitmap, timestamp) is synchronous, so the worker protects main-thread controls and drawing. Results contain landmarks, handedness, the original capture timestamp, and inference duration. The main thread ignores results older than 200 ms.

WASM executes model calculations. OffscreenCanvas supplies worker image processing without DOM. Classic-worker mode supports the loader's importScripts. A transfer list avoids copying the whole image, and finally closes it on either success or failure. [2]

CORE TECHNIQUE 3 / CONTINUITY

Fingertips and stable hand identity

Landmark	Meaning
0	Wrist
4	Thumb tip
8	Index tip
12	Middle tip
16	Ring tip
20	Little-finger tip

Figure 4. Landmark index reference. Each hand has 21 points; interaction primarily uses the five tips. These labels are illustrative, not a recognition result. [1]

```
const tips = settings.fingers === "all"
  ? [4, 8, 12, 16, 20] : [8];
const fingerId = `hand:${hand.id}:${tip}`;
```

hand.id comes from HandTracker; combining it with a landmark index gives each finger its own timer. Detection arrays can change from left/right to right/left between frames, so array indices cannot be identities.

Compare normalized wrist distances and add 0.04 for a handedness mismatch. Reuse the nearest ID below cost 0.23 if unambiguous. If the nearest two costs differ by less than 0.035, allocate a new ID.

Smooth without predicting occluded positions

```
const alpha = 1 - Math.exp(-Math.max(1, dt) / 35);
const x = old.x + (current.x - old.x) * alpha;
```

dt is milliseconds since the prior observation. A 35 ms time constant gives new positions more weight after longer gaps. With old=0.40, new=0.44, alpha=0.5, the result is 0.42. Tracks expire after 350 ms without an update; ambiguous matches do not force reuse.

CORE TECHNIQUE 4 / COORDINATES

Mapping and mirrored alignment

Model x/y are normalized to the input image; canvas logic uses CSS pixels. mapPoint shares the video contain transform and mirror. Relative z is retained but is not physical contact depth. [1]



Figure 5. A 640x480 video inside a 1000x1000 stage becomes 1000x750 with 125-pixel top/bottom margins.

```
const scale = Math.min(width / sourceWidth,
                       height / sourceHeight);
const offsetX = (width - sourceWidth * scale) / 2;
const offsetY = (height - sourceHeight * scale) / 2;
const mappedX = offsetX + (mirror ? 1 - p.x : p.x)
  * sourceWidth * scale;
const mappedY = offsetY + p.y * sourceHeight * scale;
```

In the example, the mirrored horizontal fraction 0.75 yields x=750 and centered y=500. Missing margins shifts all markers; mirroring video without coordinates reverses alignment.

keyboardGeometry finds the actual video rectangle before generating keys. DPR increases backing pixels only, while logical coordinates remain CSS pixels to avoid double scaling.

CORE TECHNIQUE 5 / SHARED GEOMETRY

Closed keys and hit testing

White-key offsets	Black-key offsets
0, 2, 4, 5, 7, 9, 11, 12	1, 3, 6, 8, 10

Figure 6. Black keys overlap white keys and take priority. MIDI 60-72 gives eight white keys and five black keys. PianoKey stores x, y, w, h, note, midi, and black; drawing and hit testing read the same data.

```
const w = width * layout.width / 100;
const h = height * layout.height / 100;
const x = (width - w) * layout.x / 100;
const y = (height - h) * layout.y / 100;
const step = w / 8;
```

Defaults are width 84%, height 30%, x=50 and y=64. x/y position within remaining free space. White keys leave a one-pixel gap on each side; black width is 58% of step and height is 61% of white-key height.

```
return p.x >= r.x - margin &&
       p.x <= r.x + r.w + margin &&
       p.y >= r.y - margin &&
       p.y <= r.y + r.h + margin;
```

All four inequalities mean the point is inside. Positive margin expands; negative contracts. The array lists white before black. hitKey reverses a copy for black priority without altering drawing order.

CORE TECHNIQUE 6 / CONFIRMED OBSERVATIONS

Hover confirmation and adaptive tolerance

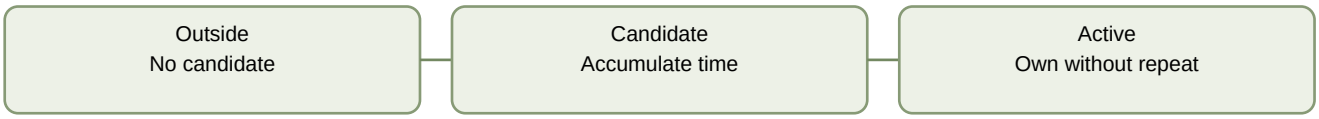


Figure 7. Entry begins confirmation; reaching dwell attacks. Exit/key change releases and requires new confirmation.

```
if (!state.interrupted) {
  state.confirmedMs += Math.min(120,
    Math.max(0, now - state.lastSeen));
}
if (target && !state.active &&
    state.confirmedMs >= this.dwell) {
  state.active = true;
  this.notes.set(finger.id, target.midi);
}
```

confirmedMs counts time supported by valid observations; active records whether this entry attacked. Default dwell is 100 ms; each update adds at most 120 ms. The ring reads confirmed time, never filling merely because the drawing clock advances.

A missing-detection example

Enter C4 at 0 ms; at 70 ms, confirmed time is 70. A miss at 100 ms freezes it. Returning at 140 ms still leaves 70. A valid observation at 175 ms raises it to 105 and attacks once. Missing hands cannot trigger from the drawing loop.

Cadence uses 0.65 * old + 0.35 * delta. Candidate grace is clamp(cadence * 2.5, 160, 320) ms; active release waits max(200, graceMs). A 150 ms valid cadence can still confirm, but frame ages above 200 ms are rejected.

Entry contracts by up to 2 pixels; holding expands by 3.5% of key width. Black-key edges do not fall through to white keys. These rules improve stability without guaranteeing perfect behavior under occlusion or low light.

CORE TECHNIQUE 7 / SHARED NOTES

Multiple fingers and chord ownership

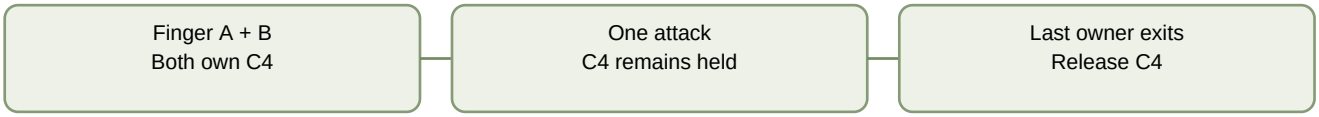


Figure 8. Shared-pitch ownership prevents one finger from interrupting another.

NoteOwners maps input IDs to MIDI pitches. Different owners may share a pitch, but only the first attacks. Fingers, mouse, and keyboard share this manager.

```
const old = this.owners.get(owner);
this.owners.delete(owner);
if (old !== undefined &&
    ![...this.owners.values()].includes(old))
  this.emit({ type: "off", midi: old });
if (midi !== undefined) {
  const occupied = [...this.owners.values()].includes(midi);
  this.owners.set(owner, midi);
  if (!occupied) this.emit({ type: "on", midi });
}
```

An unchanged owner/pitch returns before this excerpt. set(owner,midi) presses or changes; set(owner) releases. Old ownership is removed first, then new occupancy checked. main.ts forwards emit events to audio.on/off.

Transition	Event	Result
None -> A owns C4	on(60)	Start C4
A -> A and B	None	No repeat
A and B -> B	None	Keep C4
B -> none	off(60)	Release C4

Distinct pitches have independent sources, such as C4/E4/G4/B4 together. reset clears ownership before emitting releases so callbacks see an empty active set.

CORE TECHNIQUE 8 / SOUND

Piano samples and gain envelopes

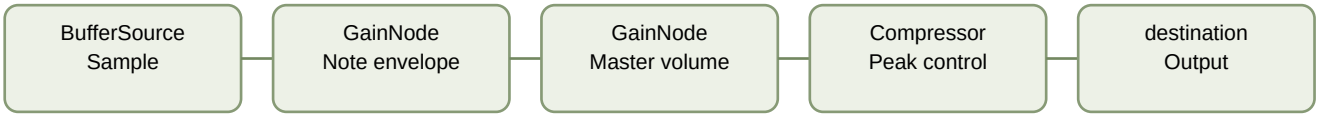


Figure 9. Per-note envelopes and master volume are separate controls.

prepare fetches five MP3 samples and decodes AudioBuffers. Commit the buffer map only after all succeed. Resume AudioContext on a user click to unlock browser audio.

```
const sample = [...this.buffers.keys()].sort(
  (a, b) => Math.abs(a - midi) - Math.abs(b - midi)
)[0];
const source = context.createBufferSource();
source.buffer = this.buffers.get(sample);
source.playbackRate.value = 2 ** ((midi - sample) / 12);
const gain = context.createGain();
gain.gain.setValueAtTime(0, context.currentTime);
gain.gain.linearRampToValueAtTime(
  0.24, context.currentTime + 0.008);
source.connect(gain);
gain.connect(this.master);
source.start();
```

Samples: C4, D#4, F#4, A4, C5 (# means sharp). A#4 is MIDI 70, nearest A4 is 69, giving rate 2^(1/12), about 1.059. An octave doubles rate. [3]

Fade 0 to 0.24 over 8 ms. On release, cancelAndHoldAtTime retains present gain; fade for 120 ms and stop after 130 ms. Every attack creates a fresh BufferSource; old callbacks cannot delete newer repeated voices.

Holding owns a naturally decaying sample without looping. Samples are Alexander Holm's Salamander Grand Piano via Tone.js Audio, CC BY 3.0; credits remain in-app and in resource notices. [4]

ENGINEERING / CONSISTENT FEEDBACK

Canvas drawing and saved settings

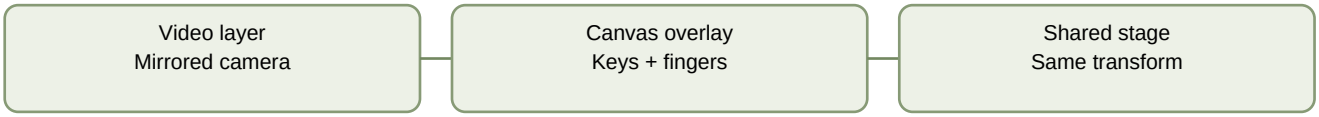


Figure 10. Video plays itself; canvas overlays keys, skeleton, and progress using the same mapping.

```
function draw(now: number) {
  ctx.clearRect(0, 0, width, height);
  if (mode === "running") {
    tracker.capture(now);
    hover.expire(now);
  }
  // Draw keys, skeleton, and fingertip progress.
  requestAnimationFrame(draw);
}
```

requestAnimationFrame schedules drawing; capture independently checks rate/busy conditions. Visual refresh can continue when inference slows. active highlights notes; candidate indicates hover, and the ring displays confirmation from 0 to 1.

Logical pixels and high-density backing

resize reads stage CSS dimensions, scales canvas backing by DPR capped at 2, and sets the transform. Fingers/keys remain in CSS pixels. Window/layout changes rebuild geometry and clear old input.

Memory and local settings

settings contains volume, delay, finger mode, skeleton, and layout. Controls update behavior first, synchronize labels, then JSON.stringify to localStorage. readSettings validates type/range; damaged JSON uses defaults. Layout/fingers/dwell reset interaction; volume/skeleton need no new geometry.

ENGINEERING / PERMISSIONS AND LIFECYCLE

Pause, resume, and actually closing the camera

Pause stops recognition, sound, and lessons, but retains MediaStream and worker. The camera light may stay on. Resume reuses the connection. Only End session stops tracks and closes worker/audio.

Action	Resources and lesson	Permission behavior
First start	Open stream; prepare model/audio	getUserMedia; browser handles permission
Either Pause	Release notes; stop frames; retain progress	No camera call; track remains live
Either Resume	Reuse stream/worker; reconfirm fingers	No new camera request
End	Tracks ended; worker/audio closed	Release device
Start after end	New session and stream	New API call; prompt depends on settings

```
function togglePause() {
  if (lesson.state === "paused" && practicing)
    void prepareLearning("resume");
  else if (mode === "running") pause();
  else if (mode === "paused") void resume();
  else if (lesson.state === "paused")
    void prepareLearning("resume");
  else if (lesson.active) suspendLearning();
}
```

Check lesson pause first: reach checking can leave recognition running while the lesson is paused. Otherwise Resume could mistakenly pause the camera. Lesson resume prepares audio and resets input without openCamera.

token, generation, and lessonGeneration isolate late permission/audio/song tasks. End invalidates tasks, stops tracks, clears srcObject, terminates worker, and closes AudioContext. Stop late-authorized streams too. Blur/background pauses; resume is explicit. Timeouts: initialization 45 seconds, inference 5 seconds.

Historical synthetic-camera verification alternated both controls twice: one getUserMedia call, identical stream, live tracks until end. This does not guarantee prompt behavior in all browsers.

TEACHING 1 / SCORES AS DATA

Song model and validation

Songs are computable steps rather than one backing MP3. SongStep uses MIDI pitch and quarter-note beats; null is a rest.

```
interface SongStep {
  midi: number | null;
  beats: number;
}
// Teaching excerpt, not a complete song (60=C4, 67=G4).
const steps = [
  { midi: 60, beats: 1 }, { midi: 60, beats: 1 },
  { midi: 67, beats: 2 }, { midi: null, beats: 1 },
];
```

Song	Notes / range	Teaching focus
Twinkle, Twinkle, Little Star	42 / C4-A4	Repeated notes and re-entry
Ode to Joy	30 / C4-G4	C-major theme; neighboring keys
Frere Jacques	32 / C4-C5	Teaching adaptation; final low G raised
Fingertip Steps	26 / C4-C5	Original; black keys and rests

The original C4-B4 keyboard had 12 keys; having a C5 audio sample did not make C5 playable on screen. The eighth white key extends the current range to MIDI 60-72.

validateSong rejects out-of-range/noninteger pitches, empty scores, invalid durations, and tempo. It never silently folds octaves. Limits: 1-1000 steps, at least one note, duration greater than 0 and at most 16 beats, tempo 30-240 BPM. practiceNotes removes rests; preview retains timing. Title, level, range, source, and adaptation are visible without claiming original-score fidelity.

TEACHING 2 / PURE STATE MACHINE

Assessment and repeated-note release gating

NoteOwners emits valid user on/off events to both PianoAudio and Lesson.input. Lesson does not inspect camera pixels or key colors. Gesture, mouse, and keyboard follow the same rules.

State	Meaning
idle	Not started; start extracts selected notes and resets statistics
playing	Wait for target on; wrong holds progress, correct consumes one step
release	Wait for held to empty; other attacks cannot advance
paused	Retain progress/time, clear ownership, await explicit resume
completed	Final success freezes results until restart/selection

```
if (this.held.has(event.midi)) return;
this.held.add(event.midi);
if (this.state === "release") return;
if (event.midi !== this.target) {
  this.mistakes++;
  this.mistakeSteps.add(this.index);
  this.attempted = true;
  return;
}
```

For C4, C4, G4: the first C4 completes one step, holding cannot complete the second. Release empties held; a new C4 attack advances again. Holding another future target cannot auto-complete it. Wrong notes still sound without advancing.

Waiting mode eventually reaches correct notes, so accuracy is firstTry / notes.length. Increment firstTry only when a step's initial attempt is correct. performance.now accumulates active time, excludes pause, and freezes at completion. Rhythm, rests, sustained duration, and chords are not graded.

TEACHING 3 / AUDIO-CLOCK SCHEDULING

Preview playback and isolation from practice

Preview and practice input are mutually exclusive. PianoAudio.schedule uses independent sources, bypassing NoteOwners and Lesson.input. Otherwise the app would grade its own playback. User input is blocked during preview; stop is always available.

```
let cursor = this.audio.time + 0.08;
const end = cursor + step.beats * 60 / song.bpm / speed;
if (step.midi !== null) {
  this.audio.schedule(step.midi, cursor,
    end - Math.min(0.06, (end - cursor) * 0.1));
}
cursor = end;
```

Mechanism	Purpose
AudioContext.currentTime	Actual note start/end reference
80 ms lookahead	Schedule a short song independently of timer jitter
25 ms UI timer	Read clock and show note/rest
Separate scheduled set	Cancel previews without confusing held user voices
Stop/song change	Cancel sources, timer, and target
Scheduling exception	catch calls stop to cancel partially queued notes

At 120 BPM one beat is 0.5 seconds; at 0.5x preview it is 1 second. speed affects timing only, not MIDI or transposition. UI options are 0.5x, 0.75x, 1x, 1.25x. Changing speed stops the old preview; clicking again restarts the chosen range.

Rests advance cursor without a source. A brief gap separates repeated notes; release fades for 120 ms. The 25 ms timer is a display updater, not a precise beat generator.

TEACHING 4 / LOCAL PRACTICE

Sections, loops, and tricky retries

UI note numbers start at 1. Internal [start,end) indices start at 0 and count non-rest notes. Validate before resetting progress. Practice and preview share currentPracticeSong.

```
let index = -1;
const steps = song.steps.filter(step => {
  if (step.midi !== null) index++;
  return index >= start && index < end &&
    !(index === end - 1 && step.midi === null);
});
```

From C4, rest, D4, E4, selecting notes 1-2 yields C4, rest, D4. Retain internal rests/beats, excluding rests before the section or after its final note. Reversed, out-of-range, and fractional indices are invalid.

A loop waits for the last release

Save the completed round before restarting. Only when ownership is empty, looping is checked, and allowLoop is true does Lesson.start run. Each round saves separately. suspendLearning clears allowLoop; clearingNotes marks cleanup so pause/song-change off events cannot restart a loop.

Case	Rule
Final note held	Show completion; await all releases
Pause/song change/end	Cancel loop intent before release
No mistakes	Hide tricky-retry button
Mistaken steps	Set records each step once
Retry tricky section	First through last mistake plus one neighboring note per side where possible

This is location-based practice, not phrase-structure analysis. Scattered errors can produce a long section. Manual phrase markers are possible future work, not an implemented claim.

TEACHING 5 / TRUSTED DISPLAY OF LOCAL DATA

Practice history and validation

On completion save song ID, date, note range, mistakes, first-try count, and active time to air-piano:history:v1. Keep 50 rounds and display 10, without accounts, server, or uploads.

```
if (lesson.state !== "completed" || recorded) return;
recorded = true;
history = [{
  songId: selectedSong.id, at: Date.now(),
  start: sectionStart, end: sectionEnd,
  mistakes: lesson.mistakes, firstTry: lesson.firstTry,
  elapsedMs: lesson.elapsedMs,
}, ...history].slice(0, 50);
localStorage.setItem(HISTORY_KEY, JSON.stringify(history));
```

The final off event and redraw still occur after completion. recorded prevents duplicate saves, resetting for a new song/round. A loop release is not a new result.

Validation	Purpose
JSON and array shape	Corruption falls back to empty history without blocking play
Known song and valid range	Reject obsolete/tampered references
Count/time/date bounds	Filter negative, fractional, and invalid data
50-record cap and field allowlist	Bound storage and discard extra fields
textContent	Stored text cannot execute as HTML
Matching song/range comparisons	Do not compare two-note and 42-note accuracy directly

Clear history without changing volume/layout. Blocked storage still allows practice with feedback. Reload does not restore incomplete progress. Local records are not tamper-proof exam results.

FEEDBACK / EXPLAIN WHAT IS WAITING

Targets, diagnostics, and key reach

Signal	Source / meaning
Blue outline and arrow	Lesson target or preview note; not an actual press
Green key	Confirmed NoteOwners.active input
Fingertip ring	confirmedMs / dwell; missing observations do not grow it
Orange error outline	Wrong note for 500 ms without advancing
Next-five strip	Separate preview, without highlighting all future keys

Actionable input reasons

Every 150 ms, check preparation, pause, preview, audio/mute, camera, delay, no hands, candidate, active, and outside state in order. Explain that a decayed active note needs exit/re-entry. A hand outline alone does not mean the index marker center is inside a key.

Optional diagnostics

Expanded details show cadence, grace, inference time, frame age, stale-result count, attack/timeouts, fingertip note, and confirmed time. Reason strings distinguish changes, exits, and timeouts. No frames or diagnostic history are stored.

Check left, middle, and right reach

With camera enabled, Check key reach asks for C4, G4, C5 in order. It keeps the lesson paused while accepting gestures. Mouse/keyboard cannot pass it. Adjust layout sliders if needed, stop anytime, then resume the lesson. It is guidance, not automatic camera calibration: no coordinate/depth estimation or automatic key enlargement.

Keep practice focused

Filter by title, level, range. Practice/preview collapses library/introduction and shortens the stage. Options/history are expandable. Back to library clears unfinished progress while retaining completed history.

VERIFICATION / EVIDENCE VERSUS EXPERIENCE

Automation and historical performance

Check	Historical record
Unit tests	36: tolerance, repetition, sections, scores, timing, storage
Browser checks	19: real model, full song, loop, song changes, permission reuse
Integrity	15 resource hashes plus MediaPipe version
Build	TypeScript and Vite; bundled static runtime
Replay	Original 12-key app on October 20, 2026; not a current rerun

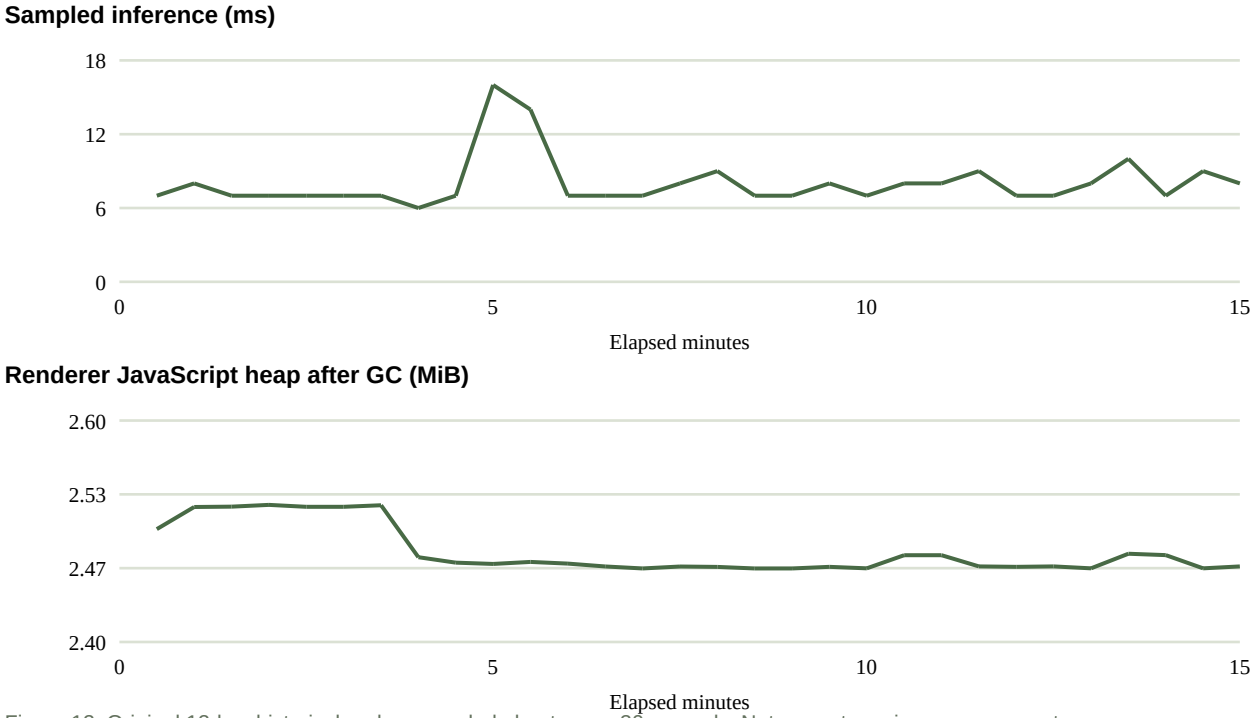


Figure 12. Original 12-key historical replay, sampled about every 30 seconds. Not current-version measurements.

Recorded environment: Windows 10.0.26200, Intel Core Ultra 9 275HX, Edge 155.0.4283.62, headless browser. Official hand-image video used real model and Web Audio. Samples: 30-31 FPS, 6-16 ms stable inference, roughly 2.47-2.52 MiB renderer heap after GC, with no sustained increase. MiB uses 1024 squared bytes; worker/GPU memory is not fully covered.

These are not human accuracy or motion-to-sound latency. Tests verify decoding/nonzero samples/source start; human chords, physical camera, speakers, and latency still need manual checks. The October 22 local environment had two live-tracking failures and much slower inference; see the acceptance record.

DEMONSTRATION / ABOUT TWELVE MINUTES

Classroom sequence and discussion

Time	Action	Explain
1 min	C4, exit, repeat	2D hit; one attack per entry
2 min	Architecture/finger/map diagrams	Model provides points; project supplies rules
2 min	Hover, misses, diagnostics	Confirmed observations, bounded grace, no missing-hand attack
2 min	First two Twinkle C4 notes	Release gate prevents automatic repeats
2 min	Short preview and loop	Separate preview; speed changes timing only
1 min	Results/history	First-try accuracy, active time, matching ranges
1 min	Pause/resume/end	Retain connection on pause; end closes it
1 min	Evidence/limits	Historical 36/19 coverage; human testing pending

Before presenting, run npm run dev in desktop Chrome/Edge. Check volume, permission, lighting, samples. Use index fingers first and check C4/G4/C5 reach. If recognition is unstable, demonstrate lesson rules with mouse/keyboard and explain that this is not gesture-stability acceptance.

Prepare Twinkle notes 1-2 (both C4). Play wrong D4, then C4 twice with release between them. First-try result should be 1/2 = 50%. Enable loop and show that it restarts only after the final release.

Three distinctions for questions

No custom model was trained: MediaPipe is pretrained; the project implements interaction, teaching, audio, and orchestration. Blue is a target, green confirmed input, ring confirmation progress. Pause does not close the camera; end stops tracks.

DELIVERY BOUNDARIES / NEXT STEPS

Limitations, improvements, and references

RGB tracking is sensitive to occlusion, low light, and fast motion. Crossed hands can make identities ambiguous. New IDs, reconfirmation, and timeouts reduce impact without guaranteeing every pose.

First validate all keys, 20 repeat entries, human two/four-finger chords, and 15 minutes of real playing on the target computer. Use external recording for motion-to-sound timing. Tune smoothing, dwell, and boundaries from evidence rather than impressions alone.

Teaching and sections are implemented. More octaves, user polygons, rhythm grading, and MIDI import are future possibilities. Downward presses need a separate strategy/calibration; velocity and real table contact require separate validation. Two-dimensional hits cannot simply be renamed real touch.

Question	Source entry
Coordinate alignment	core.ts mapPoint; main.ts keyboardGeometry
One attack per entry	core.ts HoverEngine.update / expire
Shared release	core.ts NoteOwners.set / reset
Sound generation	audio.ts prepare / on / off / close
Lesson advancement	lesson.ts input; main.ts renderLesson
Sections/previews	practice.ts songSection; song-player.ts play
Resource shutdown	main.ts end; tracking.ts stop

References and project evidence

[1] Google AI Edge Hand Landmarker Web: https://developers.google.com/edge/mediapipe/solutions/vision/hand_landmarker/web_js

[2] MDN Worker.postMessage and transferable objects: https://developer.mozilla.org/en-US/docs/Web/API/Worker/postMessage

[3] MDN AudioBufferSourceNode.playbackRate: https://developer.mozilla.org/en-US/docs/Web/API/AudioBufferSourceNode/playbackRate

[4] Tone.js Audio Salamander samples and licensing: https://github.com/Tonejs/audio/tree/master/salamander

Evidence: docs/acceptance-record.md, docs/development-guide.md, unit tests, and tests/browser. Functional historical records are October 21; charts use October 20 original-version data. Current local checks are recorded separately. Invite a final check: correct hit, one attack, exit release, re-entry repetition, device closure after end.